CS315 · ALGORITHMS ANALYSIS & DESIGN

# Midterm 442 الحل

12/01/2023 · ساعة ونص · 25 درجة

**محارب**

## 01 حل Q2: Fill in + True / False

| الفراغ | الجواب | السبب |
|---|---|---|
| InsertItem to dictionary | `O(1)` | حسب Lecture 3: في الـ log file نضيف في البداية أو النهاية مباشرة. |
| Bottom-up heap construction | `O(n)` | خطي في عدد العناصر. |
| key(u) ≤ __ ≤ key(w) | `key(v)` | خاصية الـ binary search tree. |
| Space of a queue (singly linked list) | `O(n)` | node لكل عنصر. |
| FindElement in binary tree | `O(h)` | `h` ارتفاع الشجرة: `O(log n)` لو متوازنة و `O(n)` في أسوأ حالة. |

| # | الجواب | السبب |
|---|---|---|
| 1 | **F** | لازم نفس البيئة ونفس المدخلات. |
| 2 | **T** | نص سلايد Lecture 1. |
| 3 | **T** | نعدّ أكبر عدد عمليات ممكن، وهذا الـ worst case. |

## 02 حل Q3: Heaps + Hash function

| | UpHeap | DownHeap |
|---|---|---|
| الهدف | يرجّع خاصية الـ heap-order بعد الإضافة. | يرجّع خاصية الـ heap-order بعد حذف الـ root. |
| متى | بعد ما نضيف مفتاح جديد في آخر الـ heap. | بعد removeMin، لما ننقل آخر مفتاح إلى الـ root. |
| كيف | نبدّل المفتاح مع أبوه ونطلع لفوق لين يوصل مكانه أو يوصل الـ root. | نبدّل المفتاح مع أصغر أبنائه وننزل لتحت لين يوصل مكانه أو يوصل ورقة. |
| الوقت | `O(log n)` | `O(log n)` |

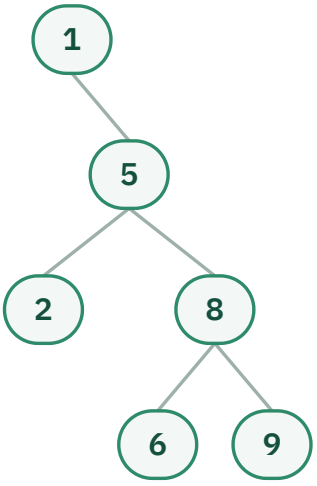
الوقت `O(log n)` لأن ارتفاع الـ heap يساوي `log n`، والمفتاح يتحرك مستوى واحد في كل تبديل.

b) الـ hash function تكون «good» لما توزّع المفاتيح على الخانات بشكل عشوائي ومتساوي فتقل الـ collisions، وتكون سريعة الحساب.

## 03 حل Q4: Trees

### a Remove node 3

العقدة 3 لها ابنين (2 و 8). نبدلها بأصغر مفتاح في الـ subtree اليمين، وهو 5 (الـ inorder successor)، ونحذف عقدة 5 الأصلية.



### b Linked structure

كل node في الشجرة تتمثل بـ object فيه ثلاث أشياء:

- الـ element.
- مؤشر على الأب (parent).
- قائمة (sequence) بمؤشرات على الأبناء.

```
R  :  parent = ∅     children = [r1, B, r2]
```

```
B  :  parent = R     children = [b1, D, b2]
```

```
D  :  parent = B     children = [d2, d3]
```

```
r1, r2, b1, b2, d2, d3 :  children = [ ]
```

## 04 Q5: Collision handling حل

| الطريقة | الشرح |
|---|---|
| **Chaining** | كل خانة تأشر على linked list فيها كل العناصر اللي جتها. بسيط، لكنه يحتاج ذاكرة إضافية خارج الجدول. |
| **Linear probing** | open addressing: نحط العنصر المتصادم في أول خانة فاضية بعدها، بشكل دائري. عيبه أن العناصر تتكتل فتطول سلسلة الـ probes. |
| **Double hashing** | open addressing: نستخدم دالة ثانية `d(k)` ونجرب الخانات `(i + j·d(k)) mod N`. لازم `d(k) ≠ 0` و `N` عدد أولي. |

## 05 حل Q6: عدّ العمليات

| السطر | operations # |
|---|---|
| currentMax ← A[0] | 2 |
| Temp ← A[0] | 2 |
| for i ← 1 to n − 1 do | 1 + n |
| if A[i] > currentMax then | 2(n − 1) |
| currentMax ← A[i] | 2(n − 1) |
| { increment counter i } | 2(n − 1) |
| return currentMax | 1 |
| **Total** | **7n** |

$$2 + 2 + (1 + n) + 2(n - 1) + 2(n - 1) + 2(n - 1) + 1 \;=\; 7n$$